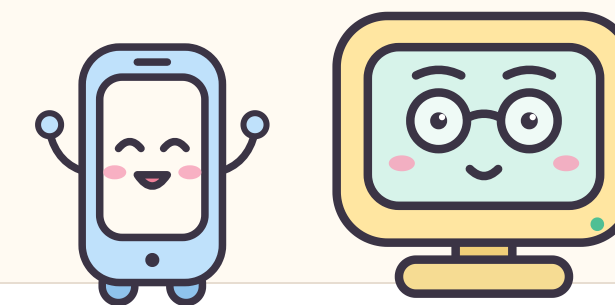




第 17 集 · 约 21 分钟

桌面②：外壳

只开一个、会弹窗、有托盘，还不拦着关机

17

从零写出 LanShare

这一集你会学到

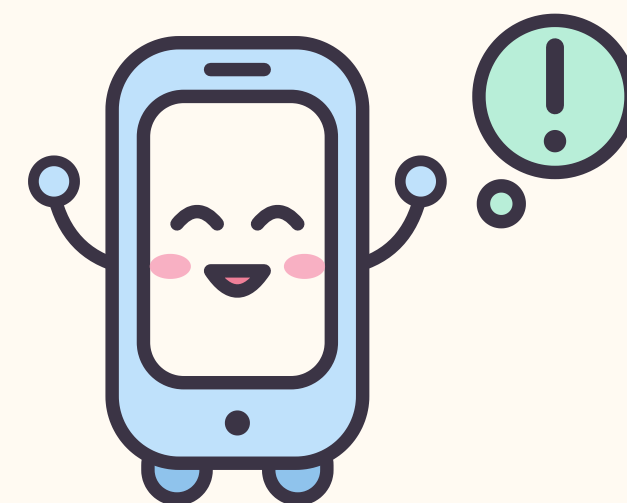
1. 命名互斥量：系统帮我们记住“已经有一个在跑了”
2. 调 Windows API：UTF-16、以 0 结尾、unsafe 和 SAFETY 注释
3. 有没有控制台，要看标准输出句柄能不能用
4. 托盘图标和 Win32 消息循环，以及关机时那句“可以”
5. 把替身换成真的，main.rs 一个字都不用改



先想一个问题

LanShare 已经在托盘里跑着，你又双击了一次图标。会发生什么？

不会起第二个。后来者发现“名字已经被占了”，读实例记录，打开第一个的主界面，然后安静地退出。





怎么知道已经有一个在跑：Python 和 Rust

Python v1 • main.py (节选)

```
1 def find_running_instance(path, timeout=1.0):
2     with open(path, encoding="utf-8") as f:
3         info = json.load(f)      # 读实例记录
4     conn = http.client.HTTPConnection("127.0.0.1", info["port"], timeout=timeout)
5     conn.request("GET", "/?k=" + info["pin"])
6     ...                          # 连得上、口令对, 才算活着
```

Rust v2 • instance.rs (预告)

```
1 let handle = unsafe { CreateMutexW(std::ptr::null(), 0, wide.as_ptr()) };
2 let already = unsafe { GetLastError() } == ERROR_ALREADY_EXISTS;
```



windows-sys：一次补齐

```
17 hyper = { version = "1", features = ["http1", "server"] }
18 hyper-util = { version = "0.1.21", features = ["tokio", "server-graceful"] }
19 if-addr = "0.13"
20 lanshare-proto = { path = "../lanshare-proto" }
21 qrcode = { version = "0.14", default-features = false, features = ["svg"] }
22 rand_core = { version = "0.6", features = ["getrandom"] }
23 serde = { version = "1", features = ["derive"] }
24 serde_json = "1"
25 socket2 = "0.6.5"
26 tokio = { version = "1", features = ["rt-multi-thread", "macros", "net", "fs", "io-util", "sync", "time", "signal"] }
27 tower = { version = "0.5", features = ["util"] }
28 windows-sys = { version = "0.61.2", features = ["Win32_Foundation", "Win32_System_Threading", "Win32_UI_WindowsAndMessaging", "Win32_System_Console", "Win32_UI_Shell",
"Win32_Networking_WinSock", "Win32_System_SystemInformation", "Win32_Storage_FileSystem"] }
29
30 [dev-dependencies]
31 reqwest = { version = "0.12", default-features = false, features = ["json"] }
32 sha2 = "0.10"
33 tempfile = "3"
34
35 [build-dependencies]
36 winresource = "0.1.31"
```



desktop/mod.rs: 换掉替身

mod.rs crates/lanshare/src/desktop/ 第 1-6 行 • 共 6 行

```
1  //! 桌面外壳：单实例、托盘、应用窗口、资源管理器、弹窗。
2
3  pub mod instance;
4  pub mod shell;
5  #[cfg(windows)]
6  pub mod tray;
```



instance.rs: 实例记录

instance.rs crates/lanshare/src/desktop/

第 21-40 行 · 共 40 行

```
21 pub fn write_record(path: &Path, record: &InstanceRecord) -> io::Result<()> {
22     if let Some(dir) = path.parent() {
23         fs::create_dir_all(dir)?;
24     }
25     let tmp = path.with_extension("json.tmp");
26     fs::write(&tmp, serde_json::to_vec(record)?)?;
27     fs::rename(&tmp, path)
28 }
29
30 pub fn read_record(path: &Path) -> Option<InstanceRecord> {
31     let record: InstanceRecord = serde_json::from_slice(&fs::read(path).ok()?).ok()?;
32     (record.port != 0 && record.key.len() == 43).then_some(record)
33 }
34
35 /// 只删本进程写的记录：别的实例写的不能动。
36 pub fn remove_record_if_own(path: &Path, pid: u32) {
37     if read_record(path).is_some_and(|r| r.pid == pid) {
38         let _ = fs::remove_file(path);
39     }
40 }
```



instance.rs: 命名互斥量

instance.rs crates/lanshare/src/desktop/ 第 65–84 行 • 共 84 行

```
65         return None;
66     }
67     Some(InstanceLock { handle })
68 }
69
70 #[cfg(not(windows))]
71 pub fn acquire(_name: &str) -> Option<InstanceLock> {
72     Some(InstanceLock {})
73 }
74 }
75
76 impl Drop for InstanceLock {
77     fn drop(&mut self) {
78         #[cfg(windows)]
79         if !self.handle.is_null() {
80             // SAFETY: handle 来自 CreateMutexW, 只在这里关闭一次。
81             unsafe { windows_sys::Win32::Foundation::CloseHandle(self.handle) };
82         }
83     }
84 }
```



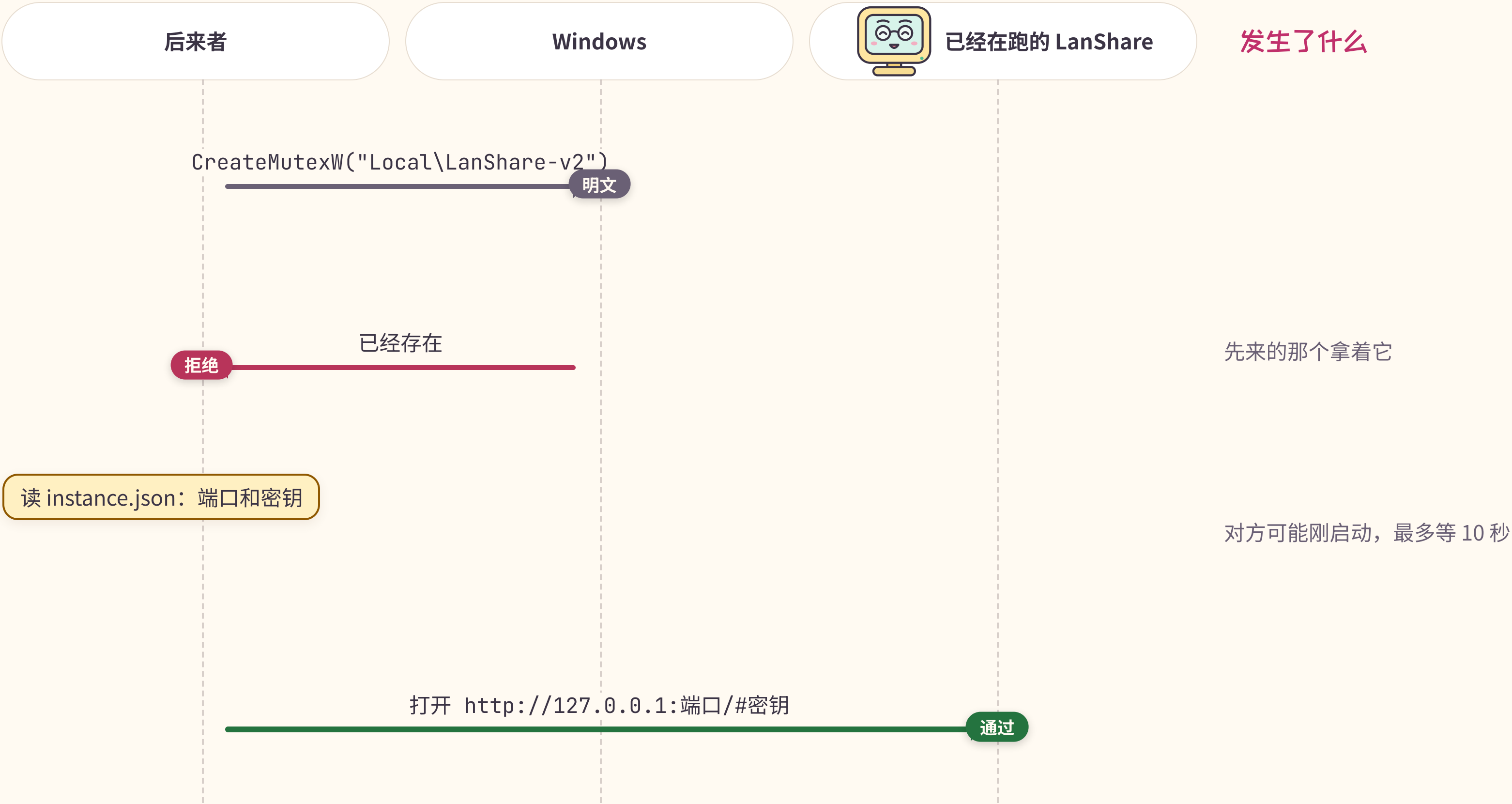
instance.rs: 测试

```
instance.rs crates/lanshare/src/desktop/ 第 107-126 行 • 共 131 行

107
108     #[test]
109     fn remove_only_own_record() {
110         let dir = tempfile::tempdir().unwrap();
111         let path = dir.path().join("instance.json");
112         write_record(&path, &record(42)).unwrap();
113         remove_record_if_own(&path, 7);
114         assert!(path.exists(), "别人的记录不能删");
115         remove_record_if_own(&path, 42);
116         assert!(!path.exists());
117         remove_record_if_own(&path, 42); // 不存在时也不报错
118     }
119
120     #[test]
121     fn named_lock_is_exclusive_until_dropped() {
122         let name = format!("Local\\LanShare-test-{}-{}", std::process::id(), line!());
123         let first = InstanceLock::acquire(&name);
124         assert!(first.is_some());
125         if cfg!(windows) {
126             assert!(InstanceLock::acquire(&name).is_none(), "第二个拿不到");
```



第二次双击之后





第一次调 Windows API，编译器说了什么

```
1 // 照着 Python ctypes 的习惯，直接传字符串：
2 MessageBoxW(std::ptr::null_mut(), "你好", "标题", MB_OK);
3 // 编译器：参数类型不对，要的是 *const u16，给的是 &str
4
5 // 转成以 0 结尾的 UTF-16，先存进变量，再取指针：
6 let text = wide("你好");
7 unsafe { MessageBoxW(std::ptr::null_mut(), text.as_ptr(), title.as_ptr(), MB_ICONERROR) };
```



shell.rs: 路径和浏览器

● ● ● shell.rs crates/lanshare/src/desktop/

第 32-51 行 • 共 51 行

```
32
33 /// 应用窗口的命令行参数（没有地址栏和标签页）。
34 pub fn app_window_args(url: &str) -> Vec<String> {
35     vec![format!("--app={url}"), "--window-size=1280,880".to_string()]
36 }
37
38 /// 资源管理器 `/select` 的参数原文。不能交给 Rust 的自动转义：explorer 有自己的一套解析规则。
39 pub fn explorer_select_arg(path: &Path) -> String {
40     format!("/select,\"{}\"", display_path(path).display())
41 }
42
43 fn explorer_exe() -> PathBuf {
44     let root = std::env::var_os("SystemRoot").map(PathBuf::from).unwrap_or_else(|| PathBuf::from(r"C:\Windows"));
45     // 绝对路径：Windows 上按名字找程序时，会先找本程序 exe 所在的目录，那里的同名程序会冒充 explorer
46     root.join("explorer.exe")
47 }
48
49 fn spawn_quiet(command: &mut Command) -> std::io::Result<()> {
50     command.stdin(Stdio::null()).stdout(Stdio::null()).stderr(Stdio::null()).spawn().map(|_| ())
51 }
```



shell.rs: 打开窗口、网址和文件夹

● ● ● shell.rs crates/lanshare/src/desktop/ 第 80-99 行 • 共 99 行

```
80
81 /// 在资源管理器里选中这个文件。
82 pub fn reveal(path: &Path) {
83     #[cfg(windows)]
84     {
85         use std::os::windows::process::CommandExt;
86         let _ = spawn_quiet(Command::new(explorer_exe()).raw_arg(explorer_select_arg(path)));
87     }
88     #[cfg(not(windows))]
89     shell_open(&path.parent().unwrap_or(path).to_string_lossy());
90 }
91
92 pub fn open_folder(path: &Path) {
93     #[cfg(windows)]
94     {
95         let _ = spawn_quiet(Command::new(explorer_exe()).arg(display_path(path)));
96     }
97     #[cfg(not(windows))]
98     shell_open(&path.to_string_lossy());
99 }
```



shell.rs: 弹窗和控制台

● ● ● shell.rs crates/lanshare/src/desktop/

第 123–142 行 • 共 142 行

```
123         unsafe { AttachConsole(ATTACH_PARENT_PROCESS) };
124         // SAFETY: 只读取本进程的标准输出句柄。
125         let out = unsafe { GetStdHandle(STD_OUTPUT_HANDLE) };
126         !out.is_null() && out != INVALID_HANDLE_VALUE
127     }
128     #[cfg(not(windows))]
129     true
130 }
131
132 /// `server::Desktop` 的真实实现。
133 pub struct Explorer;
134
135 impl crate::server::Desktop for Explorer {
136     fn reveal(&self, path: &Path) {
137         reveal(path);
138     }
139     fn open_folder(&self, path: &Path) {
140         open_folder(path);
141     }
142 }
```



shell.rs: 测试

```
● ● ● shell.rs crates/lanshare/src/desktop/ 第 164-183 行 • 共 183 行

164         assert_eq!(find_app_browser(&c, |p| p.starts_with("y") || p.starts_with("z")), Some(PathBuf::from("y/msedge.exe")));
165         assert_eq!(find_app_browser(&c, |_| false), None);
166         let defaults = browser_candidates();
167         if let Some(first_chrome) = defaults.iter().position(|p| p.ends_with("chrome.exe")) {
168             assert!(defaults[..first_chrome].iter().all(|p| p.ends_with("msedge.exe")), "Edge 全部排在 Chrome 前面");
169         }
170     }
171
172     #[test]
173     fn app_window_has_no_address_bar_flag() {
174         let args = app_window_args("http://127.0.0.1:8000/#abc");
175         assert_eq!(args[0], "--app=http://127.0.0.1:8000/#abc");
176     }
177
178     #[test]
179     fn explorer_is_an_absolute_path() {
180         assert!(explorer_exe().is_absolute());
181         assert!(explorer_exe().ends_with("explorer.exe"));
182     }
183 }
```



加依赖：tray-icon

```
18 hyper-util = { version = "0.1.21", features = ["tokio", "server-graceful"] }
19 if-addr = "0.13"
20 lanshare-proto = { path = "../lanshare-proto" }
21 qrcode = { version = "0.14", default-features = false, features = ["svg"] }
22 rand_core = { version = "0.6", features = ["getrandom"] }
23 serde = { version = "1", features = ["derive"] }
24 serde_json = "1"
25 socket2 = "0.6.5"
26 tokio = { version = "1", features = ["rt-multi-thread", "macros", "net", "fs", "io-util", "sync", "time", "signal"] }
27 tower = { version = "0.5", features = ["util"] }
28 tray-icon = "0.26.0"
29 windows-sys = { version = "0.61.2", features = ["Win32_Foundation", "Win32_System_Threading", "Win32_UI_WindowsAndMessaging", "Win32_System_Console", "Win32_UI_Shell",
    "Win32_Networking_WinSock", "Win32_System_SystemInformation", "Win32_Storage_FileSystem"] }
30
31 [dev-dependencies]
32 reqwest = { version = "0.12", default-features = false, features = ["json"] }
33 sha2 = "0.10"
34 tempfile = "3"
35
36 [build-dependencies]
37 winresource = "0.1.31"
```





tray.rs: 图标和菜单

tray.rs crates/lanshare/src/desktop/

第 30–49 行 • 共 49 行

```
30         .with_menu(Box::new(menu))
31         .with_menu_on_left_click(false) // 左键直接打开主界面，右键才弹菜单
32         .with_tooltip(tooltip)
33         .with_icon(icon)
34         .build()
35         .map_err(|e| e.to_string())?;
36
37     let open2 = on_open.clone();
38     MenuEvent::set_event_handler(Some(move |event: MenuEvent| match event.id().as_ref() {
39         "open" => open2(),
40         "folder" => on_folder(),
41         // SAFETY: 回调在 DispatchMessageW 里、也就是消息循环所在的主线程上被调用。
42         "quit" => unsafe { PostQuitMessage(0) },
43         _ => {}
44     })));
45     TrayIconEvent::set_event_handler(Some(move |event: TrayIconEvent| {
46         if let TrayIconEvent::Click { button: MouseButton::Left, button_state: MouseButtonState::Up, .. } = event {
47             on_open();
48         }
49     })));
```



tray.rs: 消息循环

tray.rs crates/lanshare/src/desktop/

第 43–62 行 • 共 62 行

```
43     _ => {}
44     }));
45     TrayIconEvent::set_event_handler(Some(move |event: TrayIconEvent| {
46         if let TrayIconEvent::Click { button: MouseButton::Left, button_state: MouseButtonState::Up, .. } = event {
47             on_open();
48         }
49     }));
50
51     // SAFETY: 标准的 Win32 消息循环; msg 是本地变量, GetMessageW 返回 0 表示收到 WM_QUIT。
52     unsafe {
53         let mut msg: MSG = std::mem::zeroed();
54         while GetMessageW(&mut msg, std::ptr::null_mut(), 0, 0) > 0 {
55             TranslateMessage(&msg);
56             DispatchMessageW(&msg);
57         }
58     }
59     MenuEvent::set_event_handler(None:::<fn(MenuEvent)>);
60     TrayIconEvent::set_event_handler(None:::<fn(TrayIconEvent)>);
61     Ok(())
62 }
```



关机时那句“可以”

- Windows 关机前，会给每个窗口发 WM_QUERYENDSESSION：“可以结束吗？”

回答 0 就是“不行”

- v1 的托盘库 pystray，对不认识的消息一律回答 0

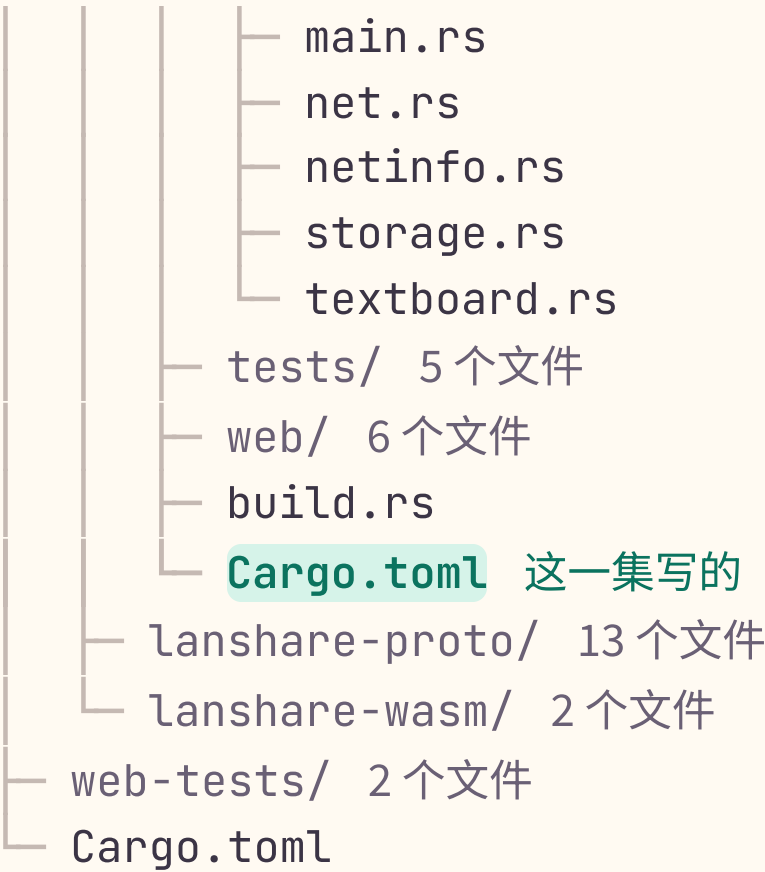
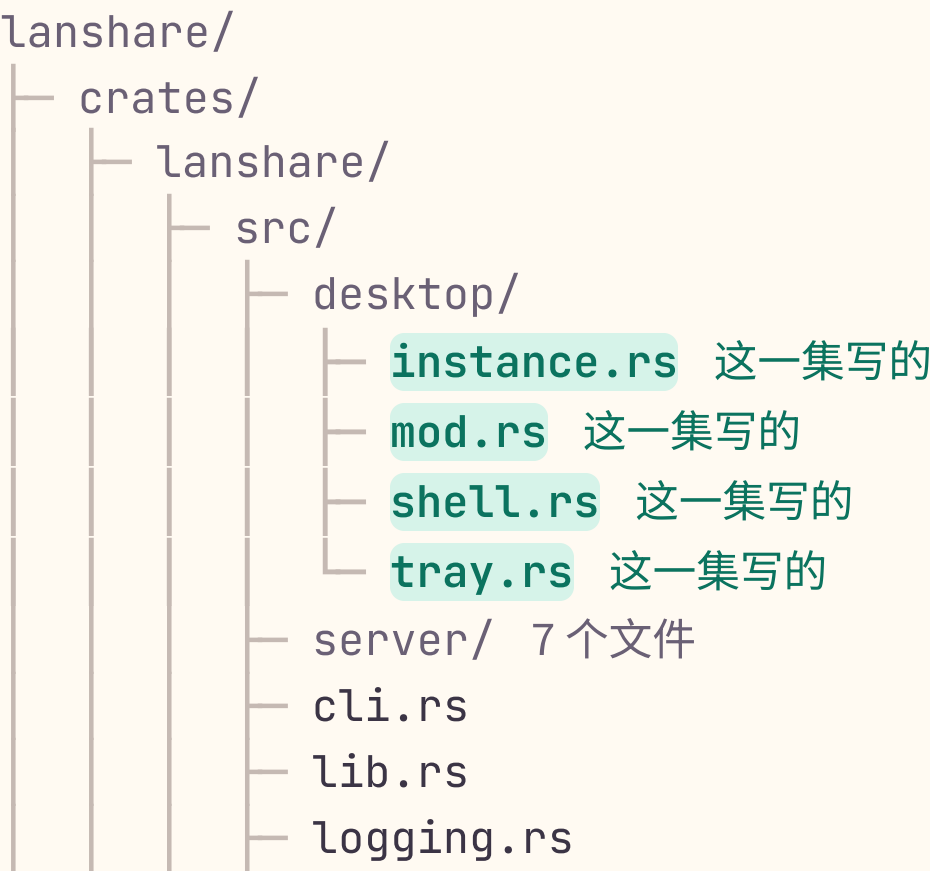
只好往它的私有字典里塞处理函数，打补丁

- v2 的 tray-icon 把不认识的消息交给系统默认处理，默认就是“可以”

光看源码不算数：真的发了一条消息，回答是 1



crates 下面只差网页了



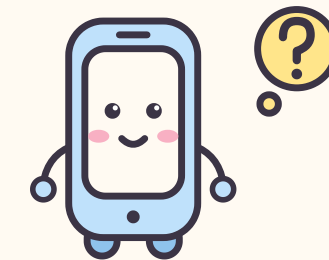


编译、跑全部测试

```
Windows PowerShell

PS D:\code\lanshare> cargo build
Updating crates.io index
Locking 75 packages to latest Rust 1.96.0 compatible versions
Adding Adler2 v2.0.1
Adding ATK v0.18.2
..... (省略 9 行, 都是 Adding)
Adding dirs v7.0.0
..... (省略 30 行, 都是 Adding)
Adding muda v0.21.0
..... (省略 25 行, 都是 Adding)
Adding tray-icon v0.26.0
..... (省略 6 行, 都是 Adding)
Compiling lanshare v2.0.0 (D:\code\lanshare\crates\lanshare)
Compiling lanshare-proto v2.0.0 (D:\code\lanshare\crates\lanshare-proto)
Compiling lanshare-wasm v2.0.0 (D:\code\lanshare\crates\lanshare-wasm)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 11.41s

PS D:\code\lanshare> cargo test --workspace
..... (省略 7 行, 都是 Compiling)
Compiling tray-icon v0.26.0
..... (省略 6 行, 都是 Compiling)
```



选择题

CreateMutexW 返回了一个有效的句柄。能不能说明“我是第一个”？

A 能，拿到句柄就是拿到了锁

B 不能，要紧接着看 GetLastError 是不是“已经存在”

C 不能，要再等 1 秒看看



选择题

CreateMutexW 返回了一个有效的句柄。能不能说明“我是第一个”？

A 能，拿到句柄就是拿到了锁

名字已被占用时它也会返回句柄，指向那个已有的互斥量。

C 不能，要再等 1 秒看看

那是 v1 探测的做法，互斥量不需要等。

B 不能，要紧接着看 GetLastError 是不是“已经存在”

名字被占用时，CreateMutexW 照样返回一个句柄。区别只能紧接着问 GetLastError，是“已经存在”就说明别人先拿到了。



选择题

最初用 “AttachConsole 成功没有” 判断有没有控制台，出了什么问题？

- A 自动化测试用管道读输出时，程序以为没有控制台，弹出对话框卡住测试
- B 双击启动时会弹出黑色窗口
- C 帮助文字会打印两遍



选择题

最初用 “AttachConsole 成功没有” 判断有没有控制台，出了什么问题？

A 自动化测试用管道读输出时，程序以为没有控制台，弹出对话框卡住测试

输出被重定向时 AttachConsole 可能失败，程序就去弹窗，测试一直等人点确定。改成看标准输出句柄能不能用，接上终端或者被重定向都算能用。

C 帮助文字会打印两遍

不会，问题出在 “该打印的时候去弹了窗”。

B 双击启动时会弹出黑色窗口

黑窗口由 windows_subsystem 决定，和这个判断无关。



选择题

下面哪一行会让托盘图标刚出现就消失？

```
1 let _ = TrayIconBuilder::new().with_icon(icon).build()?;  
2 let _tray = TrayIconBuilder::new().with_icon(icon).build()?;
```

A 第一行

B 第二行

C 两行都不会



选择题

下面哪一行会让托盘图标刚出现就消失？

```
1 let _ = TrayIconBuilder::new().with_icon(icon).build()?;  
2 let _tray = TrayIconBuilder::new().with_icon(icon).build()?;
```

A 第一行

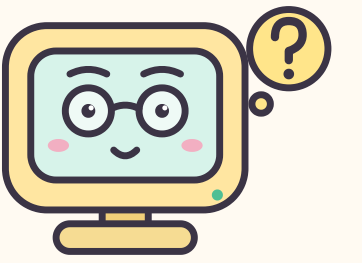
单独的下划线不绑定变量，托盘图标当场被丢弃，图标也就跟着消失了。带名字的下划线 tray 会活到函数结束。

B 第二行

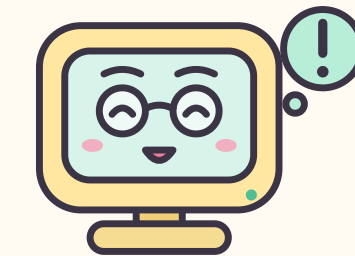
下划线 tray 是普通变量，会活到函数结束。

C 两行都不会

第一行会，而且编译器不会提醒。



实例记录为什么要先写临时文件再改名？如果直接写 `instance.json`，会出什么事？



想一想

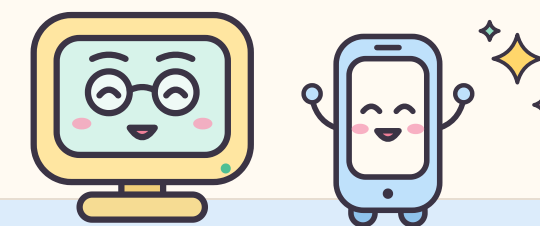
实例记录为什么要先写临时文件再改名？如果直接写 `instance.json`，会出什么事？

后来者随时可能来读。直接写的话，它可能正好读到写了一半的文件，解析失败或者读到错误的端口。先写临时文件、写完再改名，改名是一步完成的，读的一方要么看到旧的、要么看到新的完整文件，不会看到半截。



这一集写了什么

- ✓ 单实例靠命名互斥量，进程一死系统自动收回；实例记录只告诉后来者端口和密钥
- ✓ 实例记录先写临时文件再改名，读的一方永远看不到半截
- ✓ 调 Windows API：UTF-16 加结尾的 0，先存变量再取指针，每个 unsafe 写 SAFETY
- ✓ 有没有控制台，看标准输出句柄；托盘靠主线程的消息循环活着，关机时回答“可以”
- ✓ 替身换成真的，main.rs 一个字没改；crates 下面只差网页了



下一集

全课完结，感谢收看